IoT Based Smart Attendance System

with ESP32, RFID/QR Scanning and a Java Cloud Dashboard

Final Capstone Project – Part I: Proposal and Prototyping

Submitted By (Student)	
Name	Md Jahidul Islam
Student ID	IT24018
Course	Software Engineering and Project Management
Department	Information and Communication Technology
University	Mawlana Bhashani Science and Technology University

Supervised By (Teacher)	
Name	Dr. Ziaur Rahman
Designation	Professor
Department	Dept. of ICT
University	MBSTU

Date: October 26, 2026

GitHub: https://github.com/mohammadjahidulislam/IoTBasedAttendenceSystem

Abstract

Manual roll-call wastes class time and is prone to proxy attendance and recording errors. This proposal describes an IoT-based attendance system in which a classroom device built around an ESP32 microcontroller reads RFID cards (and QR codes) of students, validates the scan, and sends it over Wi-Fi using MQTT/HTTPS to a Java server. Teachers see live attendance on a dashboard and obtain reports automatically. The document presents the problem, objectives, requirements, hardware and software architecture, an Iterative-Incremental development plan with schedule, milestones, deliverables and Gantt chart, the six required UML diagrams, a risk and cost analysis, and a simple Java prototype with more than five classes taken from the class diagram.

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Objectives . . . 3
- 1.4 Scope . . . 3
- 1.5 Significance . . . 3

2 Review of Existing Approaches **3**

3 Proposed System **4**
- 3.1 Overview . . . 4
- 3.2 System Architecture . . . 4
- 3.3 Working Flow . . . 4

4 Hardware Components **4**
- 4.1 Wiring Plan (ESP32 to Modules) . . . 5
- 4.2 Device Firmware Outline . . . 5

5 Requirements **6**
- 5.1 Functional Requirements . . . 6
- 5.2 Non-Functional Requirements . . . 6
- 5.3 Technology Stack . . . 6

6 Software Process Model and Project Plan **6**
- 6.1 Chosen Model: Iterative and Incremental . . . 6
- 6.2 Work Breakdown and Schedule . . . 7
- 6.3 Gantt Chart . . . 8
- 6.4 Milestones . . . 8
- 6.5 Deliverables . . . 9

7 Risk Analysis **9**

8 UML Diagrams **9**
- 8.1 Data Flow Diagram . . . 9
- 8.2 Use Case Diagram . . . 10
- 8.3 Activity Diagram . . . 11
- 8.4 Entity Relationship Diagram . . . 12
- 8.5 Class Diagram . . . 13
- 8.6 Object Diagram . . . 13

9 Prototype **14**
- 9.1 Prototype Goal . . . 14
- 9.2 Classes Used (from the Class Diagram) . . . 14
- 9.3 Interface Layout . . . 14
- 9.4 Core Code (excerpt) . . . 14
- 9.5 Message Format (Device to Server) . . . 15
- 9.6 Repository Structure . . . 15

10 Testing Plan **16**

11 Conclusion **16**

1 Introduction

1.1 Background

Attendance is a basic academic requirement, yet most classes still use paper sheets or verbal roll-call. This takes several minutes per class, and signatures can be forged. Internet of Things (IoT) devices are cheap, small and always connected, so they are a natural fit for automating identification and reporting.

1.2 Problem Statement

- Manual attendance consumes lecture time and is error-prone.
- Proxy attendance (one student signing for another) is hard to detect.
- Consolidating paper sheets into semester reports is slow.
- Teachers get no real-time view of who is present.

1.3 Objectives

1. Design and build an ESP32-based classroom device that identifies students by RFID card or QR code.
2. Transmit each scan securely to a central server in near real time.
3. Store attendance in a relational database and prevent duplicate or invalid entries.
4. Provide a teacher dashboard with live status, history and CSV/PDF reports.
5. Keep working during short network outages by buffering scans on the device.
6. Deliver the work using a documented software process with schedule, milestones and UML models.

1.4 Scope

In scope: one device per classroom, student identification, live dashboard, reports, device management, offline buffering. **Out of scope (this phase):** face recognition, payment or library integration, native mobile apps, integration with the university ERP.

1.5 Significance

The system saves lecture time, improves data accuracy, discourages proxy attendance and provides a practical, low-cost IoT project that can be scaled to many rooms.

2 Review of Existing Approaches

Approach	**Advantage**	**Limitation**
Paper roll-call	No cost	Slow, easy to forge, hard to analyse
Biometric (fingerprint)	Hard to fake	Costly, queues, hygiene concerns
App-only QR attendance	No extra hardware	Phone sharing, GPS spoofing, needs student phones
Proposed: ESP32 + RFID/QR	Low cost, fast tap, physical presence required, real-time data	Needs Wi-Fi and cards; mitigated by offline buffer

Table 1: Comparison of attendance approaches.

3 Proposed System

3.1 Overview

A student taps an RFID card (or shows a QR code) at the classroom device. The ESP32 reads the unique ID, stamps it with time from the real-time clock, and publishes it to the server. The server validates the student against the active class session, stores the record, and sends back a result that the device shows on an OLED screen with a buzzer/LED signal. The teacher watches the dashboard update live.

3.2 System Architecture

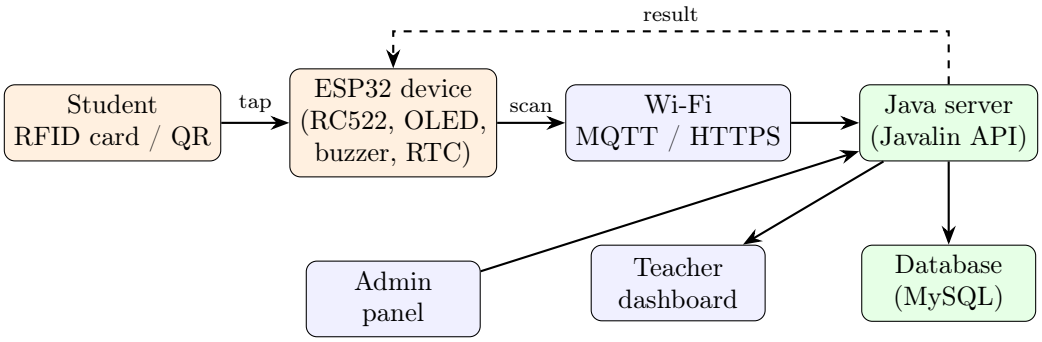


Figure 1: High-level IoT architecture.

3.3 Working Flow

1. The teacher starts a class session on the dashboard.
2. The student taps a card; the ESP32 reads the UID.
3. The ESP32 publishes `{uid, deviceId, timestamp}` to topic `attendance/scan`.
4. The server checks that the UID belongs to a student enrolled in the active course and has not already been marked.
5. An attendance record (Present or Late) is saved and the result is returned to the device.
6. The OLED shows the student's name; green LED and one beep mean success, red LED and two beeps mean rejection.
7. If Wi-Fi is down, the ESP32 stores scans in flash memory and uploads them when the connection returns.

4 Hardware Components

The component details below were collected from manufacturer and retailer product pages for the ESP32 DevKit V1 and the RC522 RFID module. Prices vary by seller and country, so the cost column is only a planning estimate that must be re-checked before purchase.

Component	Key specifications	Role	Est. cost (BDT)
ESP32 DevKit V1 (ESP-WROOM-32)	Dual-core Xtensa LX6 up to 240 MHz, 520 KB SRAM, 4 MB flash, Wi-Fi 802.11 b/g/n, Bluetooth 4.2/BLE, about 34 GPIO, 3.3 V logic	Main controller	600–900
RC522 RFID module (MFRC522)	13.56 MHz, SPI up to 10 Mbit/s, 3.3 V only, about 13–26 mA when active, reads ISO/IEC 14443-A / MIFARE cards at roughly 2–5 cm, usually sold with one card and one key tag	Student identification	150–300
RFID cards / key tags (MIFARE 1K)	13.56 MHz passive tags, unique UID per tag	Student ID	20–40 each
0.96" OLED (SSD1306)	I2C, 128×64 pixels	Status and name display	250–400
DS3231 RTC module	I2C, battery-backed accurate clock	Timestamps when offline	250–400
Buzzer and LEDs (red/green)	Active buzzer, 5 mm LEDs, resistors	Audio and visual feedback	50–100
5 V USB power supply	5 V, 1 A or more	Power	150–300
Optional GM65 QR scanner	UART barcode/QR reader module	QR code input	1200–1800

Table 2: Bill of materials with specifications (costs are estimates).

4.1 Wiring Plan (ESP32 to Modules)

Module pin	ESP32 pin	Note
RC522 SDA (SS)	GPIO 5	SPI chip select
RC522 SCK	GPIO 18	SPI clock
RC522 MOSI	GPIO 23	SPI data out
RC522 MISO	GPIO 19	SPI data in
RC522 RST	GPIO 27	Reset
RC522 3.3V / GND	3V3 / GND	Never connect to 5 V
OLED and RTC SDA / SCL	GPIO 21 / GPIO 22	Shared I2C bus
Green LED / Red LED	GPIO 25 / GPIO 26	With 220 Ω resistor
Buzzer	GPIO 33	Active buzzer

Table 3: Proposed pin mapping.

4.2 Device Firmware Outline

The firmware (Arduino C++ with the `MFRC522`, `PubSubClient` and `Adafruit_SSD1306` libraries) runs a loop: connect Wi-Fi → connect MQTT → poll RC522 → build JSON → publish → wait for result topic → show feedback. Unsent scans are saved to a small queue in flash.

5 Requirements

5.1 Functional Requirements

ID	Requirement
FR1	The device shall read the UID of an RFID card and/or decode a QR code.
FR2	The device shall send each scan with device ID and timestamp to the server.
FR3	The server shall validate that the student is enrolled in the active course.
FR4	The server shall reject duplicate scans within the same session.
FR5	The system shall mark a student Present or Late according to a configurable threshold.
FR6	The device shall give visual and audio feedback within 2 seconds.
FR7	The device shall buffer scans while offline and synchronise later.
FR8	The teacher shall start and end class sessions and view live attendance.
FR9	The admin shall register students, cards, courses and devices.
FR10	The system shall generate attendance reports per student, course and date, exportable as CSV.

5.2 Non-Functional Requirements

- **Performance:** end-to-end scan to feedback in under 2 s on a normal Wi-Fi network.
- **Security:** TLS for network traffic, hashed passwords, role-based access, per-device credentials.
- **Reliability:** no scan loss during outages up to several hours.
- **Usability:** one tap, no training required for students.
- **Maintainability:** layered Java code, documented API, version control on GitHub.
- **Scalability:** support at least 20 devices on one server.

5.3 Technology Stack

Layer	Technology
Device firmware	Arduino C++ on ESP32
Messaging	MQTT (Mosquitto broker) or HTTPS REST
Server	Java 17, Javalin, JDBC
Database	MySQL / PostgreSQL
Dashboard	HTML/JavaScript served by Javalin; JavaFX for the prototype
Documentation	LaTeX, TikZ, pgfgantt
Version control	Git and GitHub

Table 5: Technology stack.

6 Software Process Model and Project Plan

6.1 Chosen Model: Iterative and Incremental

The Iterative-Incremental model is chosen because (1) hardware and software can be delivered in small working increments, (2) early prototypes with a simulated device reduce risk while parts are being sourced, (3) feedback after each increment can correct requirements, and (4) a working system exists after the first increments even if time runs short.

Increment	Weeks	Content
I1	3–6	Core server, database, class model, device simulator, basic JavaFX interface
I2	7–9	Real ESP32 + RC522 firmware, MQTT integration, OLED/buzzer feedback
I3	10–12	Dashboard, reports, offline buffering, security
I4	13–14	Full testing, documentation, demo and final delivery

Table 6: Planned increments.

6.2 Work Breakdown and Schedule

No.	Task	Start (week)	End (week)
1	Proposal, requirements and UML design	1	2
2	Component procurement and setup	2	4
3	Database and server core (I1)	3	5
4	Java prototype and device simulator (I1)	4	6
5	ESP32 firmware and RFID reading (I2)	6	8
6	MQTT/API integration (I2)	8	9
7	Teacher dashboard and reports (I3)	9	11
8	Offline buffering and security (I3)	11	12
9	System testing and bug fixing (I4)	12	13
10	Final report, demo and release (I4)	13	14

Table 7: Project schedule.

6.3 Gantt Chart

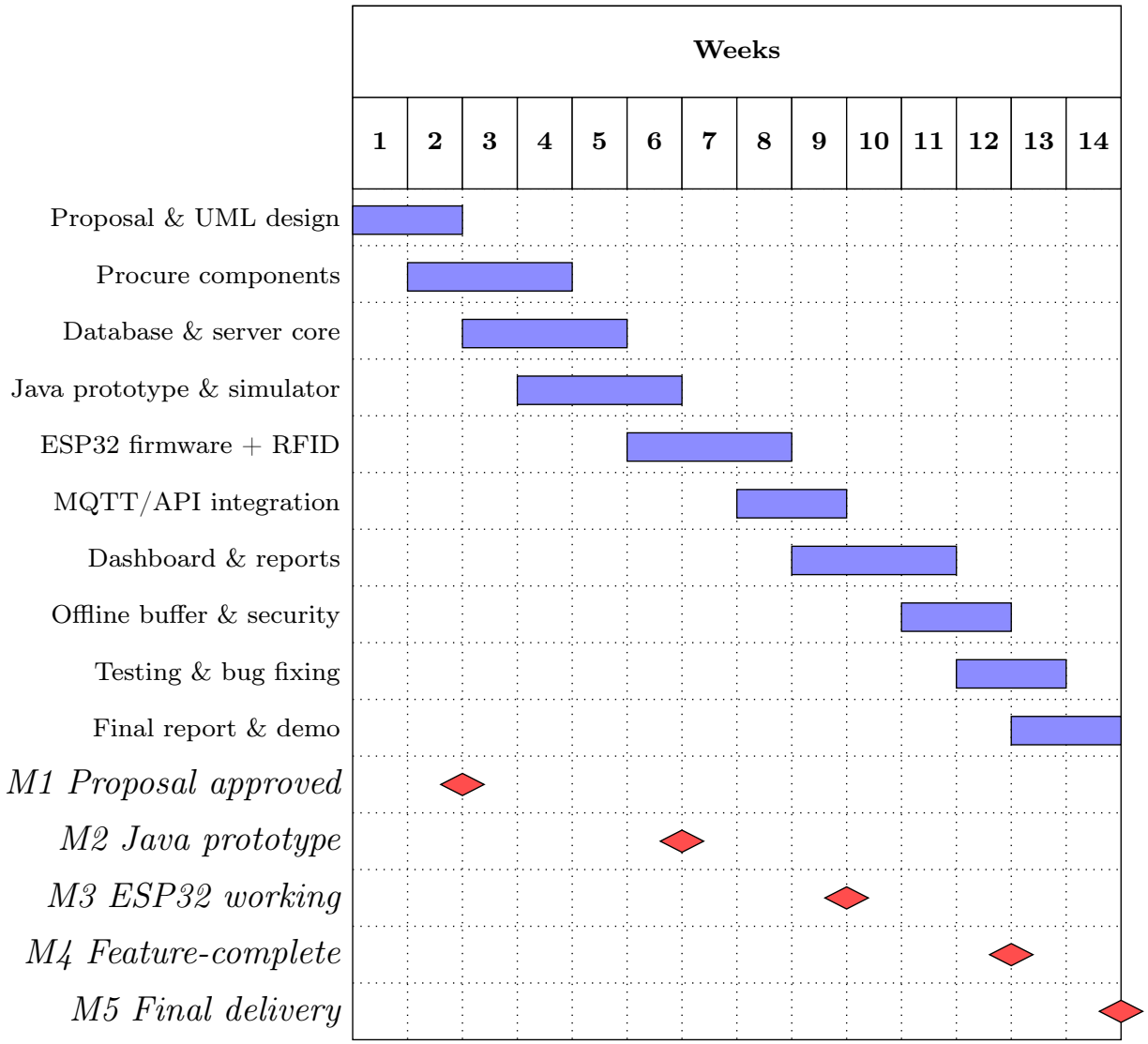


Figure 2: Gantt chart of the 14-week plan.

6.4 Milestones

ID	Week	Milestone and exit criteria
M1	2	Proposal approved: requirements and all UML diagrams complete
M2	6	Java prototype runs: simulated scan creates an attendance record
M3	9	Real ESP32 device sends an RFID scan to the server
M4	12	Feature-complete: dashboard, reports, offline sync, security
M5	14	Final delivery: tested system, report, demo and GitHub release

Table 8: Project milestones.

6.5 Deliverables

No.	Deliverable	Description
D1	Project proposal	This document with UML models (week 2)
D2	Java prototype	JavaFX interface with 5+ classes and device simulator (week 6)
D3	Device prototype	Working ESP32 + RC522 unit with wiring diagram (week 9)
D4	Server and dashboard	Source code, API documentation, database script (week 12)
D5	Test report	Test cases and results (week 13)
D6	Final report and demo	Report, slides and demo video (week 14)
D7	Repository	Tagged release on GitHub with README (week 14)

Table 9: Project deliverables.

7 Risk Analysis

Risk	Level	Impact	Mitigation
Delay in component delivery	High	Hardware milestone slips	Build the Java simulator first; order parts early
Wi-Fi outage in classroom	Medium	Lost scans	Offline buffer in device flash
Card sharing or cloning	Medium	Proxy attendance	Optional PIN/QR second factor; log device and time
Module damage (5 V on 3.3 V pins)	Medium	Replacement cost	Wiring checklist; buy a spare RC522
Scope creep	Medium	Schedule overrun	Fixed scope list; incremental delivery
Data privacy	Medium	Student data exposure	TLS, access control, minimal data stored

Table 10: Risk register.

8 UML Diagrams

8.1 Data Flow Diagram

Level 0 (Context Diagram)

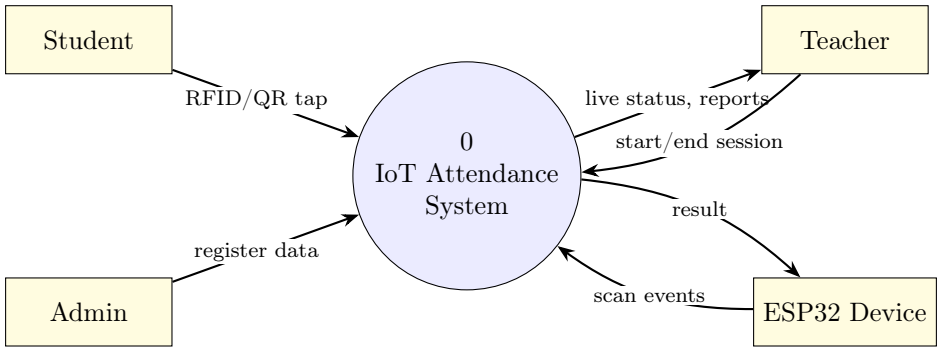


Figure 3: DFD Level 0.

Level 1

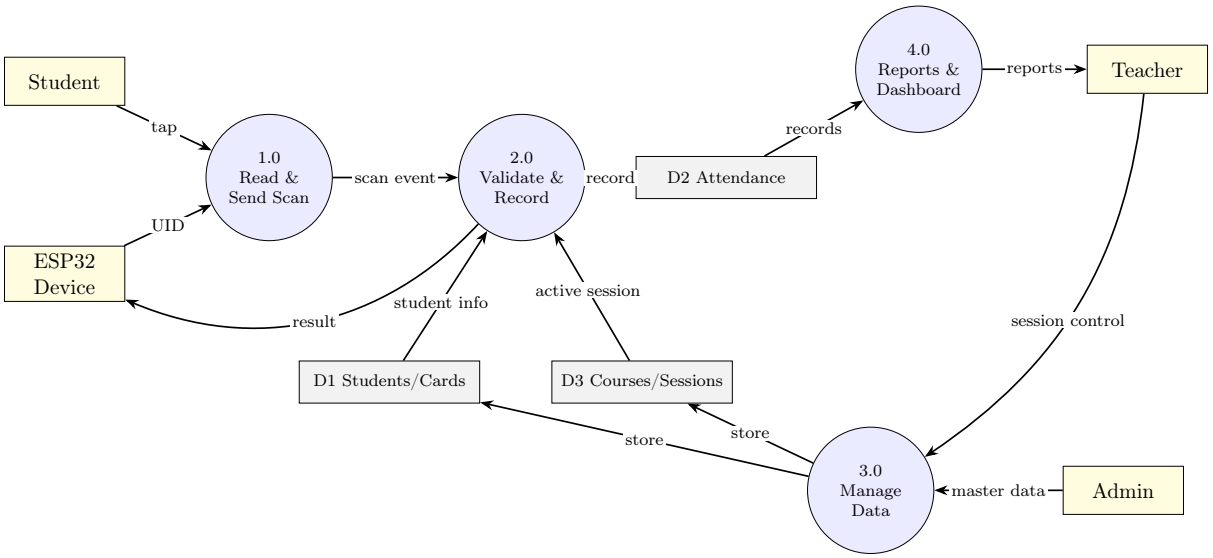


Figure 4: DFD Level 1.

8.2 Use Case Diagram

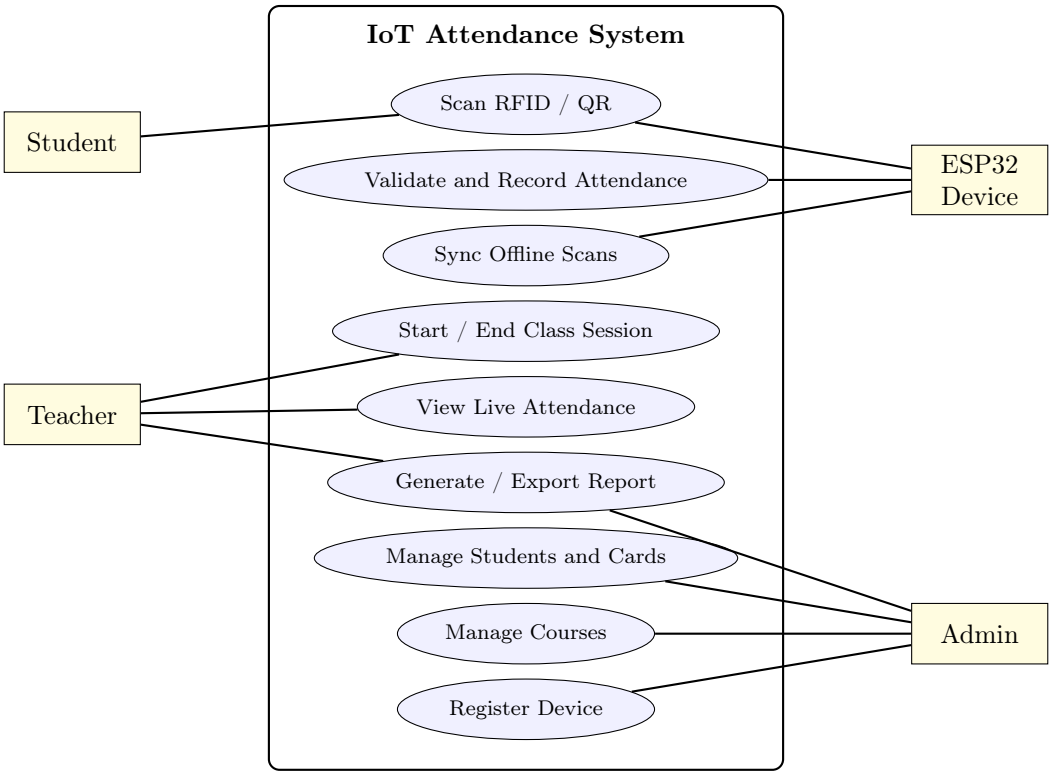


Figure 5: Use case diagram.

8.3 Activity Diagram

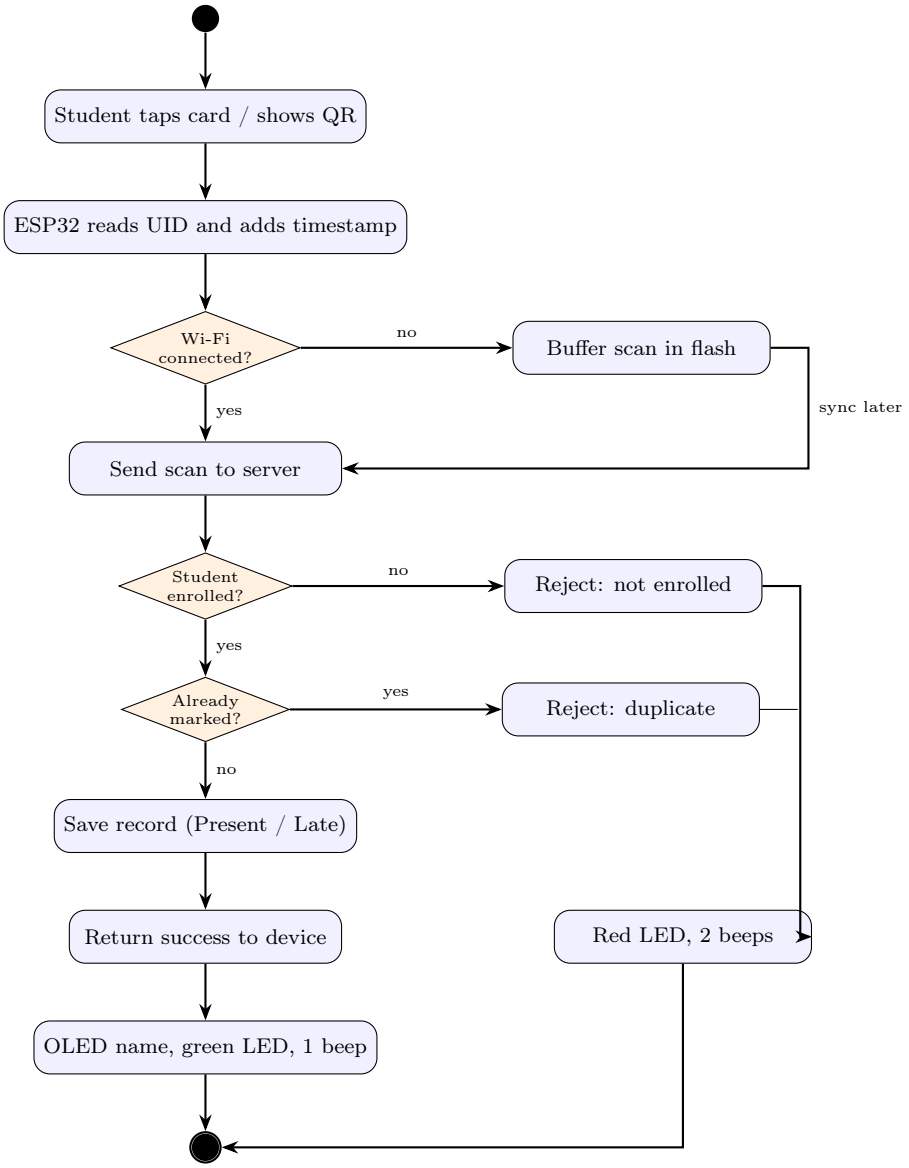


Figure 6: Activity diagram for one attendance scan.

8.4 Entity Relationship Diagram

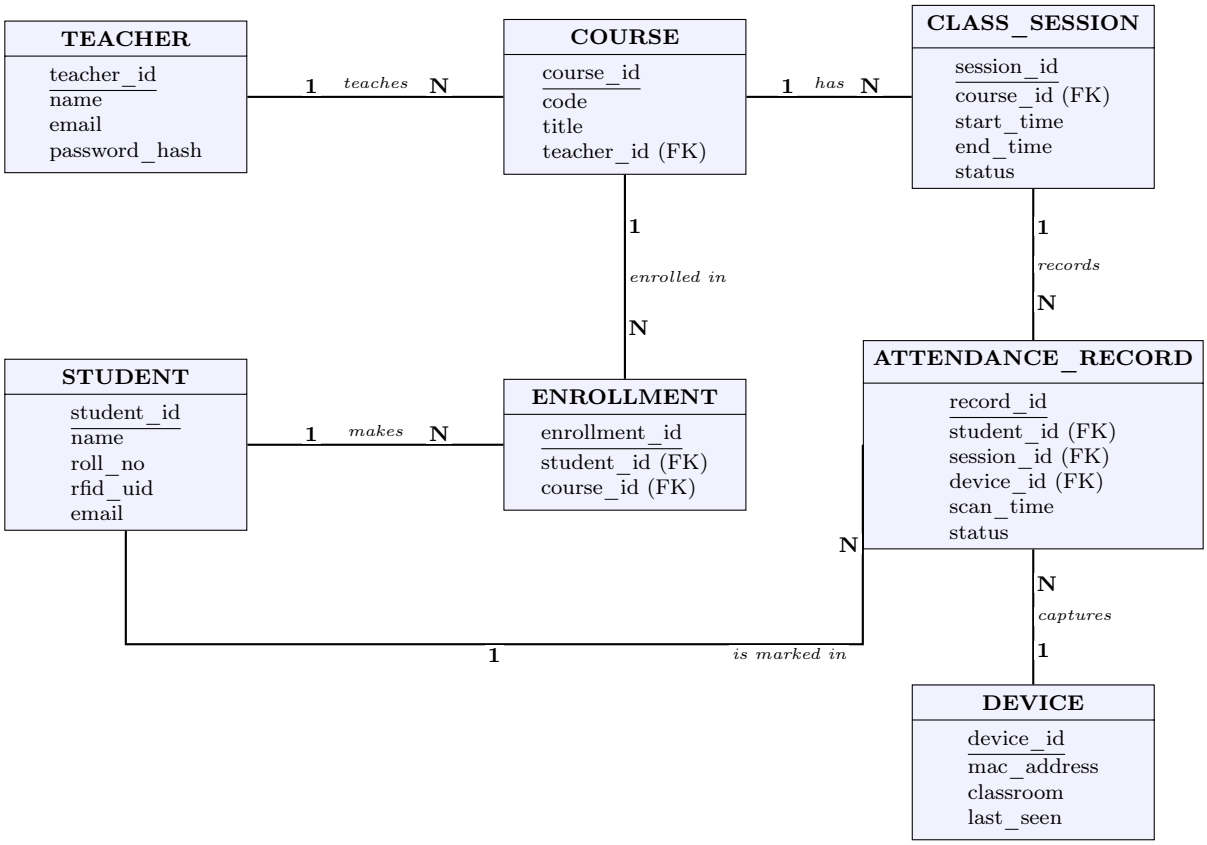


Figure 7: Entity relationship diagram (crow's-foot style cardinalities shown as 1 and N).

8.5 Class Diagram

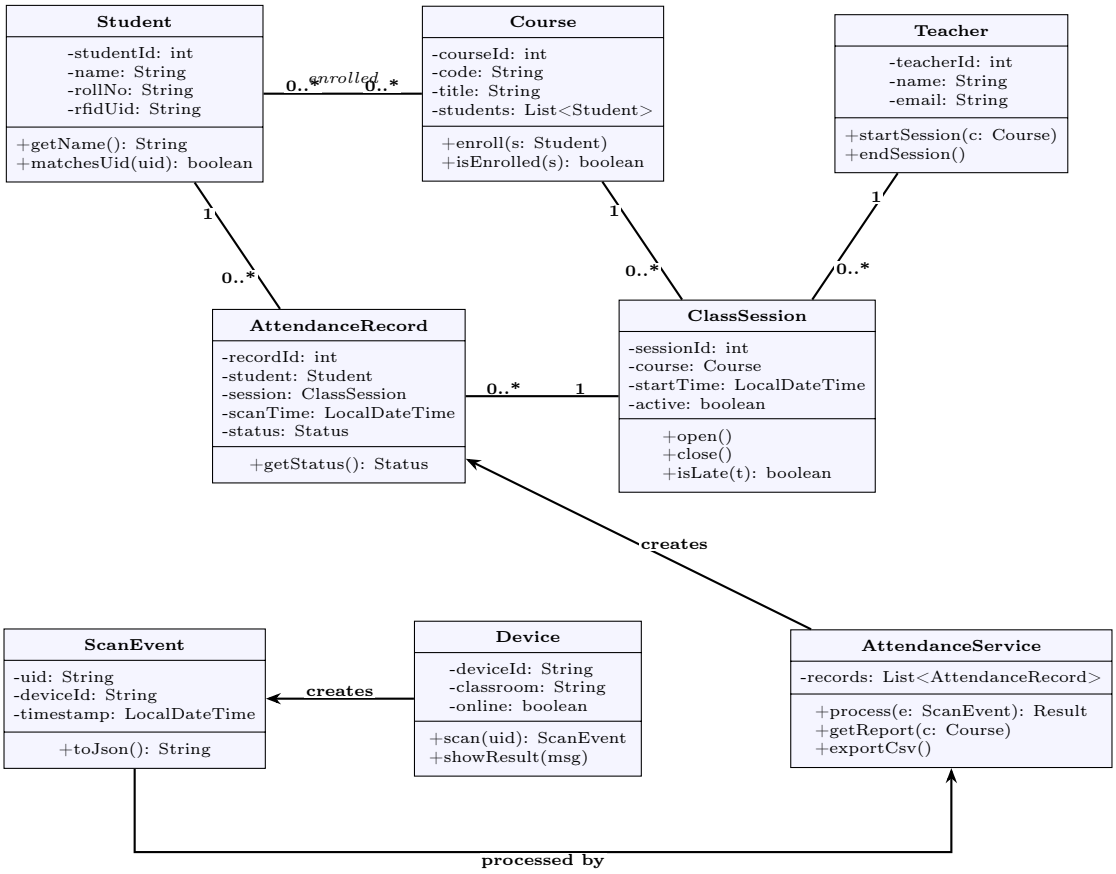


Figure 8: Class diagram.

8.6 Object Diagram

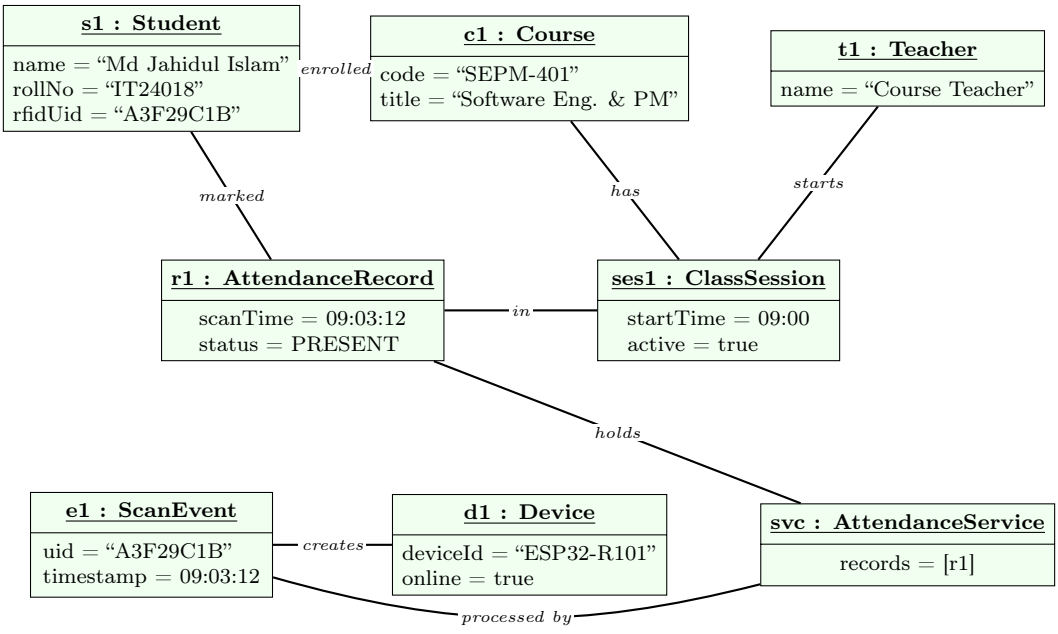


Figure 9: Object diagram for one successful scan.

9 Prototype

9.1 Prototype Goal

The first prototype demonstrates the core logic without needing hardware. A `Device` class simulates the ESP32: pressing a button in a JavaFX window (or typing a card UID) creates a `ScanEvent`, which `AttendanceService` validates and stores as an `AttendanceRecord`. The real ESP32 later replaces the simulator and sends the same JSON message, so no server logic changes.

9.2 Classes Used (from the Class Diagram)

Class	Responsibility in the prototype
`Student`	Holds name, roll number and RFID UID
`Course`	Holds the enrolled students and checks enrolment
`ClassSession`	Open/close state and late-time rule
`ScanEvent`	One scan message from a device
`Device`	Simulated ESP32 that produces scan events
`AttendanceRecord`	Stored result of a valid scan
`AttendanceService`	Validation, duplicate check, storage and report
`MainApp (JavaFX)`	Simple interface with UID field, Scan button, live table

Table 11: Prototype classes.

9.3 Interface Layout

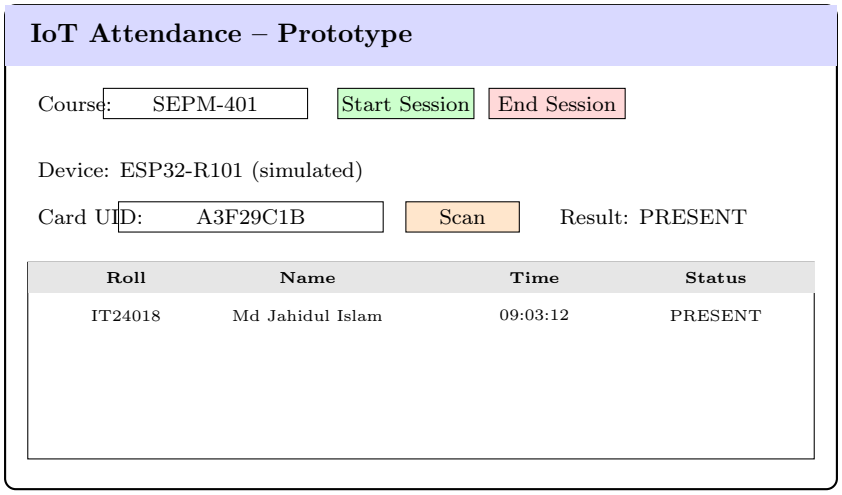


Figure 10: Planned prototype window (wireframe).

9.4 Core Code (excerpt)

Listing 1: Key classes of the prototype (simplified).

```
public class Student {
    private final int studentId;
    private final String name, rollNo, rfidUid;
    public Student(int id, String name, String rollNo, String uid) {
        this.studentId = id; this.name = name;
        this.rollNo = rollNo; this.rfidUid = uid;
    }
```

```
    public boolean matchesUid(String uid) { return rfidUid.equalsIgnoreCase(uid); }
    public String getName() { return name; }
    public String getRollNo() { return rollNo; }
}

public class ScanEvent {
    private final String uid, deviceId;
    private final java.time.LocalDateTime timestamp;
    public ScanEvent(String uid, String deviceId) {
        this.uid = uid; this.deviceId = deviceId;
        this.timestamp = java.time.LocalDateTime.now();
    }
    public String getUid() { return uid; }
    public java.time.LocalDateTime getTimestamp() { return timestamp; }
}

public class Device {
    private final String deviceId;
    public Device(String deviceId) { this.deviceId = deviceId; }
    public ScanEvent scan(String uid) { return new ScanEvent(uid, deviceId); }
}

public class AttendanceService {
    private final java.util.List<AttendanceRecord> records = new java.util.ArrayList<>();
    public String process(ScanEvent e, Course course, ClassSession session) {
        if (!session.isActive()) return "NO ACTIVE SESSION";
        Student s = course.findByUid(e.getUid());
        if (s == null) return "REJECTED: NOT ENROLLED";
        for (AttendanceRecord r : records)
            if (r.getStudent() == s && r.getSession() == session)
                return "REJECTED: DUPLICATE";
        String status = session.isLate(e.getTimestamp()) ? "LATE" : "PRESENT";
        records.add(new AttendanceRecord(s, session, e.getTimestamp(), status));
        return status;
    }
}
```

9.5 Message Format (Device to Server)

Listing 2: MQTT payload on topic attendance/scan.

```
{ "uid": "A3F29C1B", "deviceId": "ESP32-R101", "ts": "2026-11-02T09:03:12" }
```

9.6 Repository Structure

```
iot-attendance/
  docs/            proposal.tex, diagrams, gantt
  prototype-java/  src/ (Student, Course, ClassSession, ScanEvent,
                   Device, AttendanceRecord, AttendanceService, MainApp)
  firmware-esp32/  attendance.ino
  server/          Javalin API, SQL schema
  README.md
```

10 Testing Plan

ID	Type	Test case and expected result
T1	Unit	Valid UID returns PRESENT and one record is stored
T2	Unit	Unknown UID returns REJECTED: NOT ENROLLED
T3	Unit	Second scan of same card in a session returns DUPLICATE
T4	Unit	Scan after the late threshold returns LATE
T5	Integration	ESP32 scan reaches server and OLED shows result in under 2 s
T6	System	Wi-Fi off for 10 minutes: scans are buffered and synced afterwards
T7	Security	Request without valid device credentials is refused

Table 12: Test cases.

11 Conclusion

This proposal defines an IoT-based attendance system that combines an ESP32 RFID/QR device with a Java server and dashboard. The Iterative-Incremental plan delivers a working simulated prototype first and the real hardware next, which keeps the risk low. The schedule, milestones, deliverables, Gantt chart, six UML diagrams and the Java prototype design give a complete basis for implementation in the following weeks.

References

[1] Espressif Systems, *ESP32 Series Datasheet*, https://www.espressif.com.

[2] NXP Semiconductors, *MFRC522 Contactless Reader IC Datasheet*, https://www.nxp.com.

[3] OASIS, *MQTT Version 3.1.1 Specification*.

[4] I. Sommerville, *Software Engineering*, Pearson.

[5] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

[6] M. Fowler, *UML Distilled*, Addison-Wesley.